

1. Where would you rate yourself on (LLM, Deep Learning, AI, ML). A, B, C [A = can code independently; B = can code under supervision; C = have little or no understanding]

Area	Rating (A/B/C)
LLMs	B
Deep Learning	B
AI (general)	A
ML (classical)	B

2. What are the key architectural components to create a chatbot based on LLM? Please explain the approach on a high-level

Chatbot architecture is an arrangement of six functional layers that let a chatbot understand a user's message, decide what to do, and respond accordingly. Although these six functional layers are branded differently, their functions are understood similarly. LLM-powered chatbots require modular architecture with core layers such as the orchestration layer, retrieval and grounding mechanisms and memory states.

To describe each layer in further detail, the following table is presented (Layers are in order):

User → UI → Backend API → Prompt Layer → Response Post Processing → User

Layer	Role
1. User Interface (UI)	This is the web app, mobile app or chat interface that collects user inputs as well as displays responses (from the chatbot). Its main function is to pass messages forward.
2. Orchestration/Backend Layer	Handles request routing, authentication, logging and limiting. It decides what to do with each query (for example, whether to call the database or fetch data from an API for correct information). Overall, it handles coordination between rule-based/deterministic logic and generative output from the LLM.
3. Retrieval and Guarding Mechanisms	Embeds the query, searches trusted database and vector spaces and supplies the most relevant information chunks. May also use Retrieval Augmentation Generation (RAG).
4. Prompt Engineering	This structures the user query into a format suitable for the LLM (critical to ensure response quality). It includes system instructions, and context from the retrieval layer, where it dictates how those retrieved chunks are organized, tagged with sources.
5. LLM	Generates a response from system instructions by first analyzing the semantics of the user input, deducing the user's core intent, and translating the raw data from the system (backend/API) into human-like text.

6. Memory	Lets the chatbot carry context across turns, and keeps conversations within memory (both short term and long term). Allowing for more tailored responses to the user.
7. Post Processing & Safety	Filters unsafe or out-of-scope input, checks output for safety and grounding in retrieved context.

Two of these components are important to explain further. The retrieval layer helps keep the model's answers grounded in the actual data rather than relying only on what the model learned during **pretraining**. When a user asks a question, the query is converted into an embedding and compared with a vector database containing pre-embedded data of the source material. The most relevant pieces of data are then retrieved and provided to the model as context. I explain this process in more detail in Q3, since the retrieval layer is a separate subsystem.

The prompt engineering within the orchestration layer is also crucial. How you instruct the model to answer only from the provided context, and what it should do when nothing relevant was retrieved affects response quality. Therefore, the instructions given to the model should clearly state using only the provided context and permitting making up an answer. This can have a significant impact on accuracy and consistency of responses.

Lastly, guardrails are useful both before and after the model call. On the input side, they can identify prompt attempts and questions that are outside the chatbot's scope. On the output side, they can check whether the generated response is consistent with the retrieved information. I believe this is an important step, as a simple validation step can help reduce hallucinations.

3. Please explain vector databases. If you were to select a vector database for a hypothetical problem (you may define the problem) which one will you choose, and why?

A vector database indexes and stores vector embeddings for fast retrieval and similarity search. It stores data as high-dimensional numeric vectors that carry semantic information that's critical for AI to maintain long term memory, as well as draw context for complex tasks. This is the key difference with a relational database. For instance, while SQL is built for exact and range lookups on tables, a vector database is built for approximate nearest-neighbour search across dense representations of meaning.

A challenge of vector data is the complexity and scale of data that is difficult to address in traditional scalar databases, and standard vector indices. Vector databases, on the other hand, provide an effective solution through optimised storage and querying capabilities.

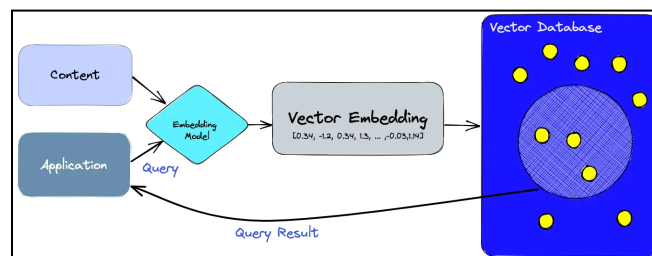


Figure: Vector Database Embedding and Retrieval Process. Source: Pinecone.in

Most vector databases use an approximate nearest-neighbour (ANN) index. HNSW (hierarchical navigable small world graphs) is the common approach today where it builds a layered graph connecting each vector to its nearest neighbours, so a search can jump through the graph toward the answer instead of checking every point. Additionally, IVF (inverted file index) clusters vectors first and then only searches the closest clusters to the vector. Although there is a tradeoff for a small amount of accuracy, it allows for speed gain.

Hypothetical problem

I want to help students find faculty in a university whose research interests match a simple-language description of what they're curious about, instead of requiring them to know the right keywords or a professor's name. Faculty research summaries, past paper abstracts, and course descriptions would each get embedded and stored as vectors. For instance, when a student says, "I'm interested in how price marketing affects consumer behaviour", the database returns the closest matches of faculty by semantics/meaning of the information rather than exact word overlap.

Choice of database

For this problem that requires multiple/hundreds of documents, and no specific budget, I'd choose Qdrant.

Below is a comparison table compiled using external research:

Database	Hosting / cost	Filtering	Best fit
Qdrant	Open source, self-hosted (free) or managed	Strong payload filtering	Small-to-mid sized projects, filtering-heavy queries
Pinecone	Managed only, paid	Good	Production systems with budget, low ops effort
Chroma	Open source, self-hosted (free)	Basic	Fast prototyping, small local projects
Weaviate	Open source or managed	Strong, GraphQL-style queries	Larger schemas with structured + vector data

Qdrant is open source and runs as a single Docker container, so there's no cost needed to get started, which matters for any medium sized project rather than one with a company's infrastructure behind it. Pinecone is the more common industrial choice and works well, but it's a paid hosted service, which can be an unnecessary overhead cost at this scale. Chroma is comparably easy to set up, however, I'd choose Qdrant for its filtering power and because it's designed to run well in production too.